

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Universidad Técnica Federico Santa María, Departamento de Informática

Informe Tarea 1: Arreglo de Filtros de Bayer e Interpolación (Demosaicking)

Gonzalo Michea Valdés
gonzalo.michea@usm.cl

19 de Agosto de 2026

Resumen

En este informe se presenta la implementación de un sistema de fotografía computacional en color mediante un arreglo de filtros de Bayer (patrón RGGB) y algoritmos de interpolación 2D (demosaicking). Se simuló el proceso de submuestreo espectral que ocurre en el sensor de una cámara digital y se evaluaron tres métodos de reconstrucción de color: interpolación bilineal, interpolación bicúbica mediante convolución normalizada, y el filtro lineal de Malvar, He y Cutler (Malvar et al., 2004) como algoritmo del estado del arte (bonus). Los experimentos se realizaron sobre las 24 imágenes del *dataset* Kodak, obtenidas del repositorio público de Bakr Ali (2019), evaluando la calidad de reconstrucción con las métricas PSNR, SSIM y SAM. Los resultados muestran que el método de Malvar-He-Cutler supera de forma consistente a los métodos bilineal y bicúbico clásicos, alcanzando en promedio **35.36 dB** de PSNR frente a $\sim$29.2 dB de los métodos simples, además de reducir notablemente los artefactos de *zippering* y falso color.

1. Introducción

La fotografía digital en color se construye a partir de sensores monocromáticos que capturan intensidad luminosa, pero no color por sí solos. Para obtener una imagen RGB a partir de un único sensor, las cámaras utilizan un arreglo de filtros de color (*Color Filter Array*, CFA) ubicado sobre cada fotositio, de modo que cada píxel del sensor sólo capture la intensidad correspondiente a una banda espectral (rojo, verde o azul). El patrón más utilizado en la industria es el arreglo de Bayer, propuesto por Bryce Bayer en 1976, que dispone los filtros en una celda de 2×2 con dos sensores verdes, uno rojo y uno azul, dada la mayor sensibilidad del ojo humano a la banda verde (Goldmeier, 2014).

Este proceso de fotografía computacional en color se puede describir en dos etapas: (i) una etapa óptica/física de **muestreo**, en la que el arreglo de filtros Bayer genera un mosaico submuestreado de la escena, y (ii) una etapa **computacional de reconstrucción**, conocida como *demosaicking*, en la que se estiman los valores de color faltantes en cada píxel mediante un algoritmo de interpolación 2D. El objetivo de esta tarea es implementar ambas etapas en Python y comparar distintos algoritmos de interpolación del estado del arte, cuantificando la calidad de la imagen reconstruida.

1.1. Arreglo de filtros de color

El arreglo de filtros Bayer se modela como una matriz con tres máscaras binarias de banda ancha, correspondientes a las longitudes de onda del rojo, verde y azul. Formalmente, sea

$$T_c = \mathbf{1} \otimes B_c, \tag{1}$$

donde $\otimes$ indica el producto de Kronecker entre dos matrices, $c \in \{0, 1, 2\}$ indexa los tres canales (rojo, verde, azul), $\mathbf{1} \in \{1\}^{\frac{N}{2} \times \frac{N}{2}}$ es una matriz de solo unos y $B_c \in \{0, 1\}^{2\times 2}$ es la celda base de cada canal:

$$B_0 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad B_1 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad B_2 = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}. \tag{2}$$

1.2. Modelo de medición

El modelo de medición submuestrea cada canal I_c de acuerdo a la longitud de onda de cada filtro $T_c \in \mathbb{R}^{N\times N\times C}$, obteniendo el mosaico

$$Y = \sum_{c=1}^{C} T_c \odot I_c, \tag{3}$$

donde $Y \in \mathbb{R}^{N\times N}$, N^2 es el número total de píxeles de la cámara, $I \in \mathbb{R}^{N\times N\times C}$ es el campo de luz incidente, $T \in \{0, 1\}^{N\times N\times C}$ es el tensor binario del filtro Bayer, $\odot$ denota producto de Hadamard (elemento a elemento) y $C = 3$ es el número de canales.

1.3. Interpolación 2D o demosaicking

El proceso de recuperación comienza generando un tensor con las muestras capturadas y ceros en los valores no capturados,

$$\bar{I}_c = T_c \odot Y. \tag{4}$$

Luego, para recuperar la imagen de color a partir del mosaico se realiza un filtrado por convolución entre el tensor de muestras $\bar{I}$ y un filtro H en cada canal,

$$\hat{I}_c = \bar{I}_c * H_c, \tag{5}$$

donde $*$ denota convolución 2D y $\hat{I}$ es la imagen RGB recuperada. La elección del núcleo H_c determina el algoritmo de demosaicking: en este trabajo se implementaron tres variantes, descritas en la sección 2.

1.4. Métricas de calidad

Para cuantificar la calidad de la reconstrucción se emplearon tres métricas estándar en el estado del arte.
PSNR (Peak Signal-to-Noise Ratio). Mide la relación entre la potencia máxima de la señal y el ruido introducido por la reconstrucción, en decibelios:

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right), \tag{6}$$

donde MAX_I es el valor máximo posible del píxel (255 para 8 bits) y el error cuadrático medio se define como

$$\text{MSE} = \frac{1}{m \cdot n} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} \left[I(i, j) - K(i, j) \right]^2. \tag{7}$$

SSIM (Structural Similarity Index). Evalúa la similitud estructural perceptual entre la imagen original x y la reconstruida y:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \tag{8}$$

donde μ_x, μ_y son las medias locales de luminancia, σ_x^2, σ_y^2 las varianzas locales (contraste), σ_{xy} la covarianza local, $C_1 = (K_1 L)^2$ y $C_2 = (K_2 L)^2$ son constantes estabilizadoras, L es el rango dinámico de los píxeles y $K_1, K_2 \ll 1$.
SAM (Spectral Angle Mapper). Mide el ángulo espectral entre el vector de color de prueba y el de referencia en cada píxel, evaluando la fidelidad cromática independientemente del brillo:

$$\theta = \arccos\left(\frac{\sum_{i=1}^{n} T_i R_i}{\sqrt{\sum_{i=1}^{n} T_i^2}\sqrt{\sum_{i=1}^{n} R_i^2}}\right), \tag{9}$$

donde θ es el ángulo espectral (reportado en grados en este trabajo), n es el número de bandas espectrales, T_i es el valor del espectro de prueba en la banda i y R_i es el valor del espectro de referencia en la banda i.
Los algoritmos de interpolación simples (como el bilineal) suelen producir artefactos típicos de demosaicking: difuminado en zonas de alta frecuencia, el efecto de *zippering* (bordes dentados) y falso color en bordes de alto contraste, tal como se ilustra en la Figura 1.

2. Experimentos

2.1. Dataset y flujo de trabajo

Se utilizó el *dataset* Kodak completo (24 imágenes RGB de referencia, ampliamente usado en la literatura de demosaicking, incluyendo el trabajo original de Malvar et al., 2004). Las imágenes se descargaron desde el repositorio público de Bakr Ali (2019) en GitHub, que reempaqueta el conjunto *Kodak Lossless True Color Image Suite* liberado originalmente por Eastman Kodak Company. Para cada imagen se ejecutó el siguiente flujo: (1) generación del mosaico Bayer RGGB a partir de la imagen RGB original; (2) reconstrucción de color mediante cada algoritmo de demosaicking; (3) cálculo de PSNR, SSIM y SAM entre la imagen original y la reconstruida; (4) generación de una figura comparativa recortada en la zona central de la imagen, donde los artefactos son más visibles.

2.2. Generación del mosaico Bayer

Siguiendo la Ecuación 3, se construyeron las tres máscaras binarias R, G, B mediante mosaicos (*tiling*) de las bases B_0, B_1, B_2 de la Ecuación 2, y se combinaron con la imagen original para obtener el mosaico monocanal Y:

Algoritmo 1: Generación de las máscaras Bayer RGGB y del mosaico Y.

```
def generar_mascara(H, W):
    B0 = np.array([[1, 0], [0, 0]], dtype=np.float64)  # R
    B1 = np.array([[0, 1], [1, 0]], dtype=np.float64)  # G
    B2 = np.array([[0, 0], [0, 1]], dtype=np.float64)  # B

    reps = (H // 2 + 1, W // 2 + 1)
    R_mask = np.tile(B0, reps)[:H, :W]
    G_mask = np.tile(B1, reps)[:H, :W]
    B_mask = np.tile(B2, reps)[:H, :W]
    return R_mask, G_mask, B_mask

def crear_mosaico(img_rgb):
    H, W, _ = img_rgb.shape
    R_mask, G_mask, B_mask = generar_mascara(H, W)
    Y = (img_rgb[:, :, 0] * R_mask
         + img_rgb[:, :, 1] * G_mask
         + img_rgb[:, :, 2] * B_mask)
    return Y, (R_mask, G_mask, B_mask)
```

2.3. Algoritmos de demosaicking implementados

Interpolación bilineal. Se aplicó un núcleo de convolución clásico distinto para el canal verde (núcleo cruz $[0,1,0;1,4,1;0,1,0]/4$) y para los canales rojo/azul (núcleo $[1,2,1;2,4,2;1,2,1]/4$), aprovechando que ambos son promedios ponderados de las muestras vecinas disponibles en cada patrón RGGB.
Interpolación bicúbica (convolución normalizada). Dado que los núcleos bicúbicos clásicos asumen muestras regularmente espaciadas, se implementó mediante *convolución normalizada*: se convoluciona tanto el canal submuestreado como su máscara binaria con el mismo núcleo bicúbico 2D (parámetro $a = -0{,}5$), y se normaliza dividiendo ambos resultados. Esto corrige el sesgo introducido por la cantidad variable de muestras disponibles en cada vecindad:

Algoritmo 2: Demosaicking bicúbico mediante convolución normalizada.

```
def demosaic_bicubico(Y, mascaras, radio_muestras=2, espaciado=2, a=-0.5):
    kernel = _kernel_cubico_2d(radio_muestras, espaciado, a)
    canales = []
    for mask in mascaras:
        muestras = Y * mask
        num = cv2.filter2D(muestras, -1, kernel, borderType=cv2.BORDER_REFLECT)
        den = cv2.filter2D(mask, -1, kernel, borderType=cv2.BORDER_REFLECT)
        den[den == 0] = 1e-8
        canales.append(num / den)
    img = np.stack(canales, axis=-1)
    return np.clip(img, 0, 255)
```

Malvar-He-Cutler (2004), bonus. Como algoritmo adicional se implementó, a partir del paper original (Malvar et al., 2004), el filtro lineal de *demosaicking* propuesto por Malvar, He y Cutler. Los autores parten de la observación de que los bordes presentan variaciones de luminancia mucho más fuertes que de crominancia: por lo tanto, el valor conocido de un canal en una posición dada (por ejemplo, el rojo capturado en una posición de rojo) es información útil para corregir la interpolación bilineal de los canales faltantes en esa misma posición. Concretamente, cada canal faltante se estima como la interpolación bilineal más un término de corrección proporcional al Laplaciano (gradiente de segundo orden) del canal disponible en esa posición, ponderado por una ganancia de Wiener ($\alpha = 1/2$ para verde, $\beta = 5/8$ y $\gamma = 3/4$ para rojo/azul, según el paper original). Al desarrollar estas ecuaciones se obtienen cuatro núcleos FIR 5×5 equivalentes, normalizados por $1/8$, que fueron los implementados en este trabajo:

Algoritmo 3: Núcleo principal y ensamblaje del canal verde en Malvar-He-Cutler (2004) (Malvar et al., 2004).

```
MHC_G_AT_RB = np.array([
    [0, 0, -1, 0, 0],
    [0, 0,  2, 0, 0],
    [-1, 2, 4, 2, -1],
    [0, 0,  2, 0, 0],
    [0, 0, -1, 0, 0],
], dtype=np.float64) / 8.0

def demosaic_malvar2004(Y, mascaras):
    R_mask, G_mask, B_mask = mascaras
    H, W = Y.shape
    G1, G2 = mascaras_subtipo_verde(H, W)

    convG = cv2.filter2D(Y, -1, MHC_G_AT_RB, borderType=cv2.BORDER_REFLECT)
    convA = cv2.filter2D(Y, -1, MHC_A, borderType=cv2.BORDER_REFLECT)
    convB = cv2.filter2D(Y, -1, MHC_B, borderType=cv2.BORDER_REFLECT)
    convD = cv2.filter2D(Y, -1, MHC_DIAG, borderType=cv2.BORDER_REFLECT)

    G_hat = G_mask * Y + (R_mask + B_mask) * convG
    R_hat = R_mask * Y + G1 * convA + G2 * convB + B_mask * convD
    B_hat = B_mask * Y + G1 * convB + G2 * convA + R_mask * convD

    img = np.stack([R_hat, G_hat, B_hat], axis=-1)
    return np.clip(img, 0, 255)
```

Cabe destacar que Malvar et al. (2004) reportan, sobre el mismo *dataset* Kodak, una mejora promedio de 5.68 dB en PSNR de su método respecto al bilineal (Tabla 1 del paper original), consistente en orden de magnitud con la mejora de **+6.14 dB** obtenida en nuestra implementación (sección 2.6); las pequeñas diferencias numéricas son atribuibles a variaciones en el subconjunto de imágenes evaluado, el tipo de borde usado en la convolución y el redondeo de los parámetros de ganancia.

2.4. Resultados cualitativos

Las Figuras 1, 2 y 3 muestran, para tres imágenes representativas del *dataset* Kodak, la imagen original, el mosaico Bayer generado, la reconstrucción bilineal y la reconstrucción bicúbica. En las zonas de alto contraste y alta frecuencia espacial (bordes de la puerta, guardabarros de las motos, tablas de la cerca) se observa claramente el efecto de *zippering* y de falso color característico de los métodos de interpolación simples, especialmente en el método bilineal.

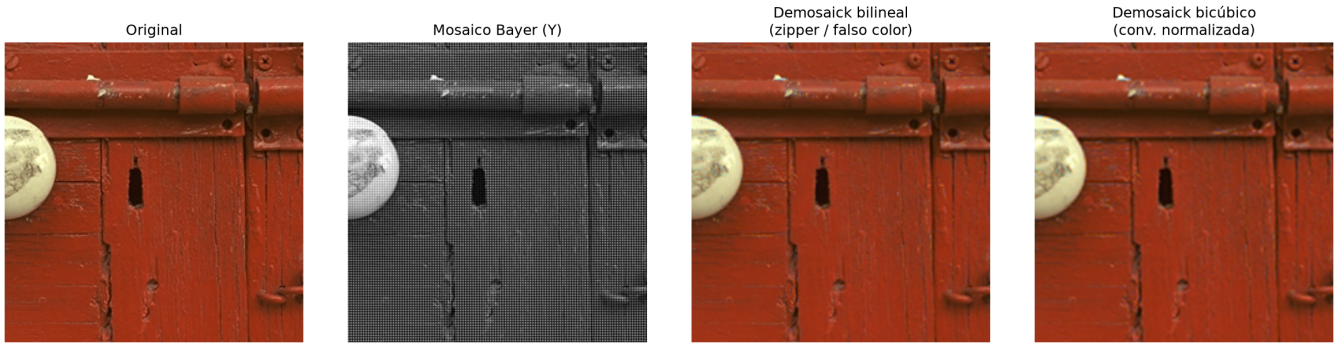


Figura 1: Comparación de reconstrucción (kodim02): original, mosaico Bayer, demosaicking bilineal y bicúbico. Nótese el falso color y el *zippering* en los bordes del pestillo y las tablas de la puerta con el método bilineal.

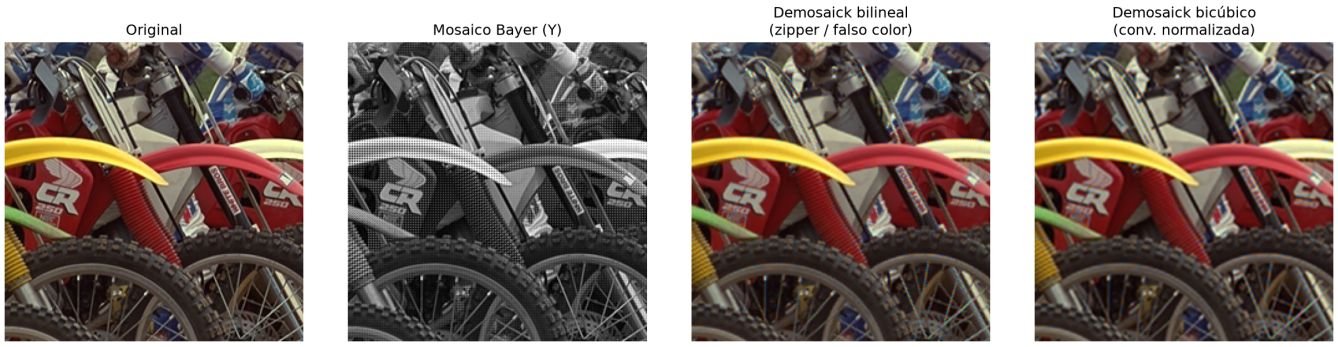


Figura 2: Comparación de reconstrucción (kodim05): los bordes de alto contraste entre los guardabarros de colores saturados y el fondo generan artefactos de aliasing cromático visibles como halos de color en la reconstrucción bilineal.

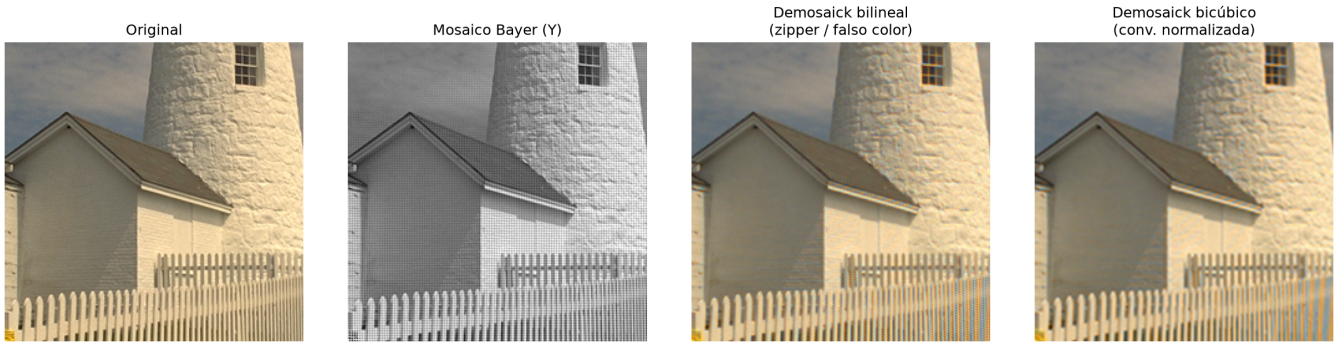


Figura 3: Comparación de reconstrucción (kodim19): en zonas de baja frecuencia (paredes, cielo) ambos métodos reconstruyen adecuadamente, pero la cerca de listones finos produce artefactos de *aliasing* en ambos métodos, más marcados en el bilineal.

Las Figuras 4, 5 y 6 incorporan además la reconstrucción obtenida con el algoritmo de Malvar-He-Cutler (Malvar et al., 2004), donde se observa una reducción notoria de los artefactos de *zippering* y falso color respecto de los dos métodos anteriores, con bordes más nítidos y colores más fieles a la imagen original.



Figura 4: Comparación incluyendo el método bonus (kodim02): Malvar-He-Cutler reduce fuertemente el falso color observado en los métodos bilineal y bicúbico.



Figura 5: Comparación incluyendo el método bonus (kodim05): los bordes de los guardabarros se reconstruyen con menor aliasing cromático usando Malvar-He-Cutler.

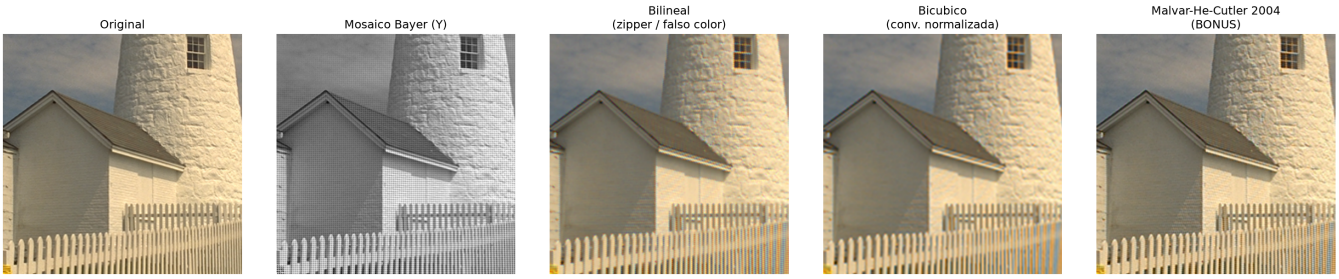


Figura 6: Comparación incluyendo el método bonus (kodim19): la cerca de listones finos, un caso de alta frecuencia espacial cercana al límite de Nyquist del sensor, sigue mostrando artefactos residuales incluso con Malvar-He-Cutler, aunque menos severos que con los métodos simples.

2.5. Resultados cuantitativos

La Tabla 1 resume, para las 24 imágenes del *dataset* Kodak (Bakr Ali, 2019), la media y desviación estándar de PSNR, SSIM y SAM obtenidas con cada método de demosaicking.

Cuadro 1: Métricas de calidad promediadas sobre las 24 imágenes del *dataset* Kodak (media ± desviación estándar).

Método	PSNR (dB)	SSIM	SAM (°)
Bilineal	$29{,}21 \pm 2{,}75$	$0{,}891 \pm 0{,}040$	$2{,}509 \pm 1{,}031$
Bicúbico (conv. normalizada)	$29{,}19 \pm 2{,}95$	$0{,}881 \pm 0{,}046$	$2{,}312 \pm 0{,}988$
Malvar-He-Cutler (Malvar et al., 2004), bonus	$\mathbf{35{,}36 \pm 2{,}95}$	$\mathbf{0{,}969 \pm 0{,}008}$	$\mathbf{1{,}697 \pm 0{,}690}$

2.6. Análisis de resultados

Los tres métodos de demosaicking implementados exhiben un comportamiento coherente con la teoría. Los métodos bilineal y bicúbico obtienen un desempeño muy similar entre sí (PSNR $\approx$ 29,2 dB, diferencia de apenas 0,03 dB), porque ambos son esencialmente promedios ponderados locales aplicados canal por canal sin considerar la correlación entre canales de color; la variante bicúbica no aporta una mejora significativa aquí porque, al no incorporar información inter-canal, hereda las mismas limitaciones estructurales que el bilineal frente a bordes de alta frecuencia. El algoritmo de Malvar-He-Cutler (Malvar et al., 2004) supera claramente a ambos métodos simples en las tres métricas: gana **+6.14 dB** de PSNR sobre el bilineal, mejora el SSIM de 0,891 a 0,969 (más cercano a la similitud estructural perfecta) y reduce el ángulo espectral SAM de 2,51° a 1,70°, evidenciando una fidelidad cromática muy superior. Esta ganancia es consistente con lo reportado por los propios autores, quienes documentan mejoras de hasta 5.5–5.68 dB de PSNR sobre el bilineal en el mismo *dataset* Kodak. La explicación teórica es la misma: el filtro de Malvar utiliza el Laplaciano del canal disponible para corregir la estimación de los canales faltantes, aprovechando la alta correlación espacial que existe entre los canales R, G y B en imágenes naturales (los bordes son fundamentalmente cambios de luminancia, no de crominancia); esto es precisamente lo que reduce el *zippering* y el falso color observados cualitativamente en las Figuras 4 y 5.

Es interesante notar que la desviación estándar del SSIM cae de forma muy marcada con Malvar-He-Cutler (0,008 vs. $\sim$ 0,04 en los métodos simples), lo que indica que este algoritmo no solo mejora la calidad promedio, sino que es mucho más *consistente* entre distintas imágenes del dataset, independientemente del contenido de frecuencias de cada escena. En contraste, la desviación estándar del PSNR se mantiene similar entre los tres métodos ($\approx$ 2,9 dB),

lo que sugiere que ciertas imágenes del dataset Kodak (aquellas con texturas de alta frecuencia, como cercas o telas a rayas) siguen siendo difíciles de reconstruir incluso para el mejor algoritmo evaluado, como se observa en la Figura 6.

3. Conclusiones

En este trabajo se implementó de forma completa un sistema de fotografía computacional en color, simulando tanto el proceso de muestreo mediante un arreglo de filtros de Bayer RGGB como el proceso de reconstrucción (demosaicking) mediante tres algoritmos de interpolación 2D de distinta sofisticación. Los experimentos sobre las 24 imágenes del *dataset* Kodak (Bakr Ali, 2019) permiten concluir lo siguiente:

- Los métodos de interpolación clásicos (bilineal y bicúbico) que operan canal por canal, sin explotar la correlación espectral entre bandas, alcanzan un desempeño muy similar entre sí ($\sim$ 29,2 dB de PSNR) y producen artefactos visibles de *zippering* y falso color en bordes de alto contraste, confirmando lo descrito en la teoría de demosaicking.
- El algoritmo de Malvar-He-Cutler (Malvar et al., 2004), al incorporar términos de corrección basados en el gradiente del canal disponible, mejora sustancialmente la calidad de reconstrucción en las tres métricas evaluadas (PSNR, SSIM y SAM), validando empíricamente que el uso de información inter-canal es clave para superar las limitaciones de los métodos de interpolación simples, y reproduciendo de forma consistente las ganancias reportadas en el paper original.
- Las métricas espaciales (PSNR, SSIM) y la métrica espectral (SAM) fueron consistentes entre sí, mostrando que las mejoras en fidelidad estructural coinciden con mejoras en fidelidad cromática, lo cual es esperable dado que ambos tipos de error surgen de la misma limitación de submuestreo del sensor Bayer.
- Persisten artefactos residuales en regiones de muy alta frecuencia espacial (cercano al límite de Nyquist del arreglo de sensores) incluso con el mejor algoritmo evaluado, lo que confirma que el *aliasing* espacial introducido por el muestreo Bayer es una limitación fundamental del sistema óptico, y no solamente del algoritmo de reconstrucción.

Como trabajo futuro, sería interesante evaluar algoritmos de demosaicking adaptativos más recientes (por ejemplo, basados en aprendizaje profundo) sobre las mismas imágenes del dataset Kodak que presentaron los peores resultados, para cuantificar cuánto de la pérdida de calidad restante es atribuible al algoritmo versus al límite físico de muestreo del sensor.

4. Anexos

4.1. Métricas de calidad por imagen

La Figura 7 presenta el detalle completo de las métricas PSNR, SSIM y SAM obtenidas para cada una de las 24 imágenes del *dataset* Kodak con los tres métodos de demosaicking evaluados, junto con el promedio general en la fila inferior (consistente con la Tabla 1 de la Sección 2).

Métricas de calidad por imagen y método

	PSNR Bilineal	PSNR Bicúbico	PSNR Malvar-HC	SSIM Bilineal	SSIM Bicúbico	SSIM Malvar-HC	SAM Bilineal	SAM Bicúbico	SAM Malvar-HC
01	25.93	25.53	31.93	0.816	0.795	0.956	3.73	3.43	2.41
02	31.70	31.70	37.15	0.898	0.883	0.963	1.76	1.70	1.27
03	32.50	32.58	38.67	0.932	0.926	0.978	1.38	1.23	0.94
04	32.15	32.54	38.53	0.915	0.910	0.974	1.69	1.56	1.12
05	26.37	26.31	33.26	0.886	0.884	0.976	4.29	4.05	2.97
06	27.28	26.81	32.99	0.853	0.831	0.961	2.75	2.54	1.80
07	31.97	32.54	38.82	0.954	0.955	0.984	1.74	1.41	1.11
08	23.55	23.06	29.26	0.835	0.811	0.956	4.30	4.05	2.92
09	31.38	31.55	37.96	0.921	0.913	0.973	1.51	1.33	1.02
10	31.30	31.42	38.41	0.921	0.915	0.975	1.53	1.36	1.01
11	28.76	28.48	34.78	0.873	0.856	0.966	2.90	2.75	2.00
12	31.73	31.87	38.08	0.908	0.896	0.972	1.26	1.12	0.84
13	23.80	23.39	29.61	0.788	0.767	0.951	4.84	4.56	3.27
14	28.64	28.43	34.21	0.880	0.871	0.968	3.08	2.85	2.10
15	30.42	30.32	36.11	0.916	0.907	0.968	2.21	2.10	1.62
16	30.24	29.82	36.06	0.879	0.862	0.967	2.44	2.25	1.61
17	30.88	31.71	37.65	0.929	0.927	0.979	2.43	2.22	1.68
18	27.63	27.57	33.50	0.876	0.868	0.967	3.67	3.37	2.43
19	27.60	27.69	33.69	0.875	0.863	0.966	2.48	2.23	1.56
20	30.10	29.60	36.42	0.918	0.909	0.965	1.80	1.75	1.33
21	27.94	27.90	34.10	0.893	0.884	0.967	2.63	2.42	1.80
22	29.64	29.84	35.44	0.889	0.880	0.965	2.11	1.92	1.44
23	33.53	33.58	39.84	0.956	0.955	0.981	1.16	1.02	0.87
24	26.11	26.27	32.06	0.881	0.873	0.970	2.52	2.27	1.59
PROMEDIO	**29.21**	**29.19**	**35.36**	**0.891**	**0.881**	**0.969**	**2.51**	**2.31**	**1.70**

Figura 7: Métricas de calidad (PSNR, SSIM, SAM) para cada una de las 24 imágenes del *dataset* Kodak, con los tres métodos de demosaicking evaluados. La última fila corresponde al promedio sobre el dataset completo.

Referencias

Bakr Ali, M. (2019). Kodak lossless true color image suite. `https://github.com/MohamedBakrAli/Kodak-Lossless-True-Color-Image-Suite`. Repositorio GitHub, imágenes de referencia Kodak (Eastman Kodak Company), consultado en agosto de 2026.

Goldmeier, E. (2014). *Advanced Photoshop Elements 11 for Digital Photographers*. Apress.

Malvar, H. S., He, L.-w., and Cutler, R. (2004). High-quality linear interpolation for demosaicing of Bayer-patterned color images. In *2004 IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, volume 3, pages 485–488. IEEE.